

- Laboratori 05 -

Indústria 4.0 / Societat 5.0

Simulació

Q1 2025/2026

Javier Zhangpan

Laura Adell Caballero

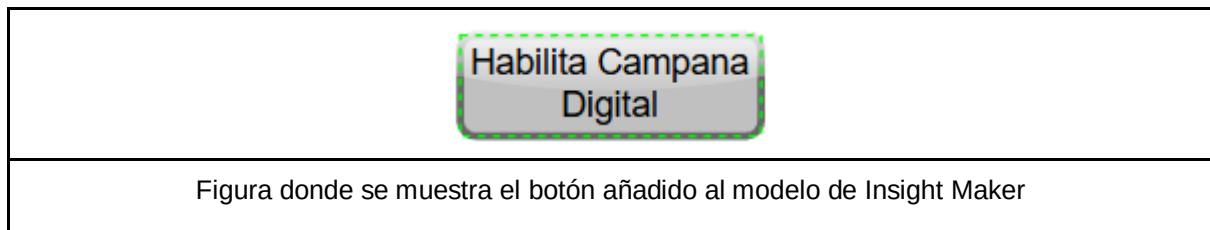
1. Conexión con Insight Maker.....	3
2. Automatización de las simulaciones.....	6
3. Conexión con entorno de visualización.....	12
4. Visualización 2D.....	16
5. Pantallas	22

1. Conexión con Insight Maker

Primero de todo, antes de ir a la implementación de los bloques en Snap, tenemos que podernos conectar a nuestro modelo en Insight Maker:

<https://insightmaker.com/insight/6Z5wpdR5bMfHlj5WbRSyJO>

Para poder hacer esto hemos añadido el siguiente botón en el modelo:



A partir de este botón, el cual es una réplica del que estaba en el ejemplo, establecemos una conexión WebSocket para interactuar con un broker MQTT a través del servidor remoto broker.emqx.io:8084 de manera que maneja la conexión entre Insight Maker y Snap y mantiene la actividad del WebSocket con un mecanismo de ping para evitar desconexiones por inactividad. Así permitimos, a través de las funciones de evaluación de código dinámico eval(m), procesar mensajes recibidos y generar respuestas en forma de código JavaScript.

Una vez ya tenemos el modelo preparado para conectarse, también hemos de preparar Snap para ello.



Figura donde se muestran los bloques para conectarse desde Snap al modelo de Insight Maker

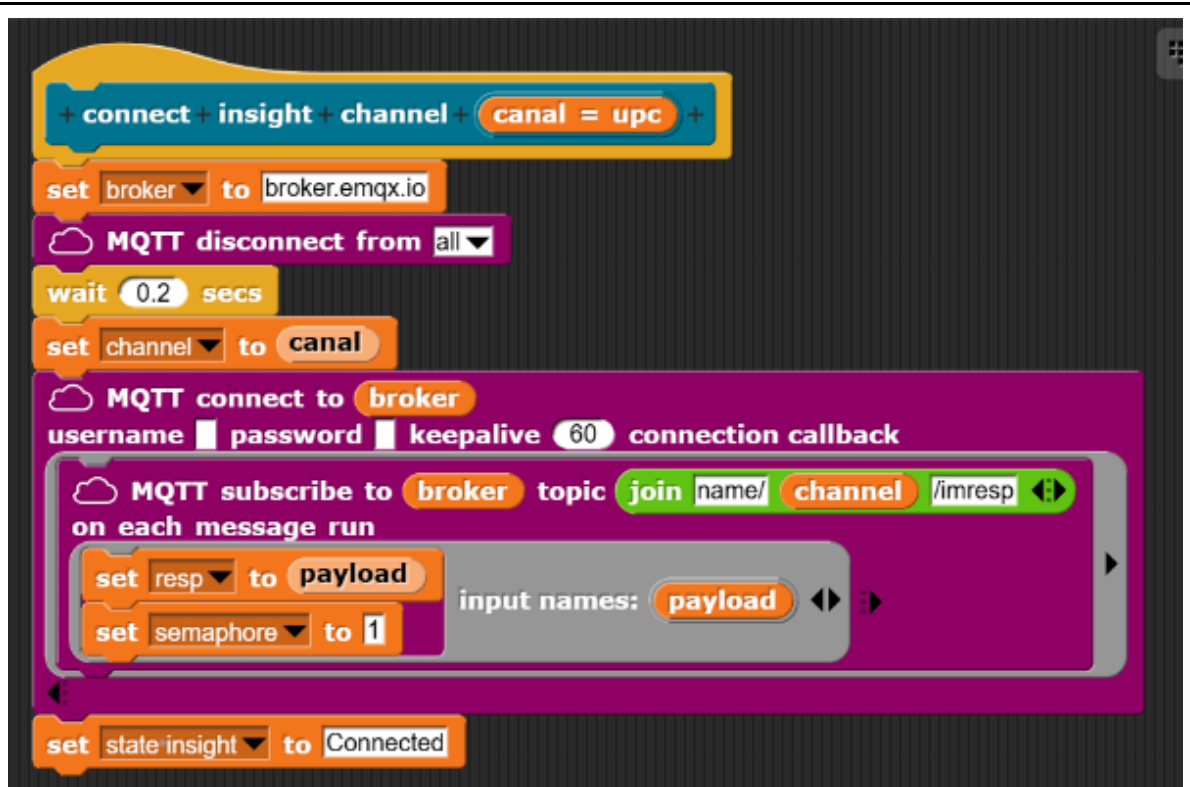


Figura donde se muestra la estructura interna del bloque "connect insight channel"

El bloque permite establecer una conexión MQTT a través del servidor remoto broker.emqx.io:8084 en el canal indicado por el usuario.

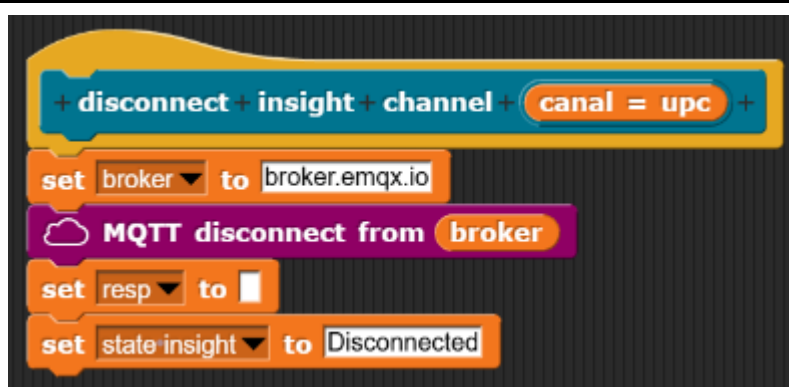


Figura donde se muestra la estructura interna del bloque "disconnect insight channel"

Finalmente, se nos permite desconectarlos de una conexión MQTT del servidor remoto `broker.emqx.io:8084`.

2. Automatización de las simulaciones

En este apartado explicaremos brevemente cómo hemos implementado los bloques necesarios para automatizar las simulaciones. Para ello, también adjuntamos capturas de pantalla para mejorar la legibilidad.

En primer lugar, se nos pedía la capacidad de modificar los parámetros de la simulación. Por este motivo, y basándonos en los bloques proporcionados como base de la práctica, hemos implementado una serie de bloques set y get de los diferentes parámetros.

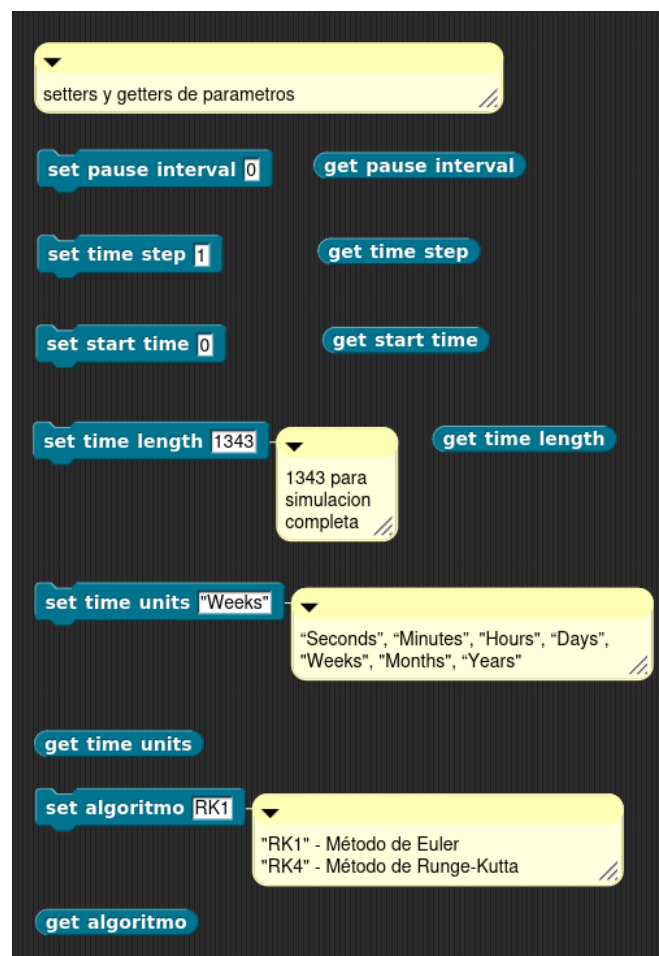


Figura donde se muestran los bloques de set y get de parámetros de simulación

A continuación adjuntamos un ejemplo de la implementación interna del bloque, ya que todos son muy parecidos.

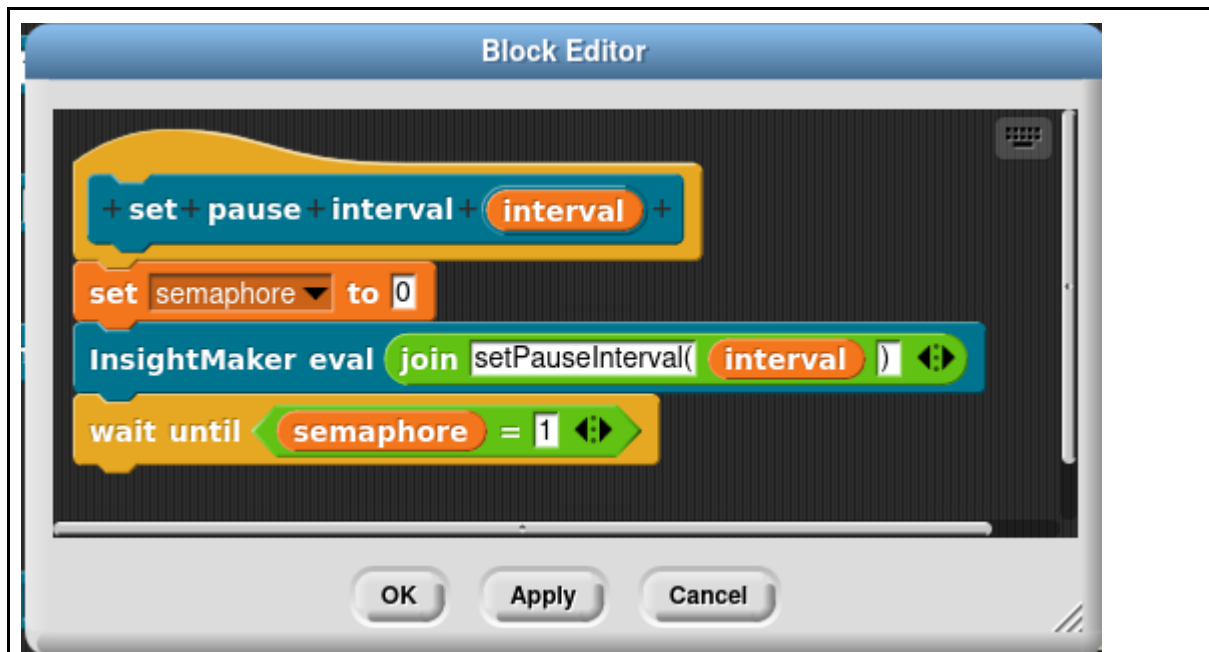


Figura donde se muestra la estructura interna del bloque "set pause interval"

Podemos destacar que el bloque recibe un parámetro "interval", envía una llamada a la API de InsightMaker usando "interval" y se espera al *callback* de la API. Desafortunadamente, no hemos añadido validación de parámetros, aunque sí que hemos añadido comentarios en las funciones con parámetros válidos no triviales de deducir.

En segundo lugar, se nos pedía la capacidad de ejecutar la simulación bajo diferentes escenarios. Esto lo hemos interpretado como la necesidad de tener un bloque que ejecute la simulación y establezca los parámetros a la vez. A continuación se muestra el bloque en cuestión y su estructura interna.



Figura donde se muestra el bloque que ejecuta la simulación con parámetros

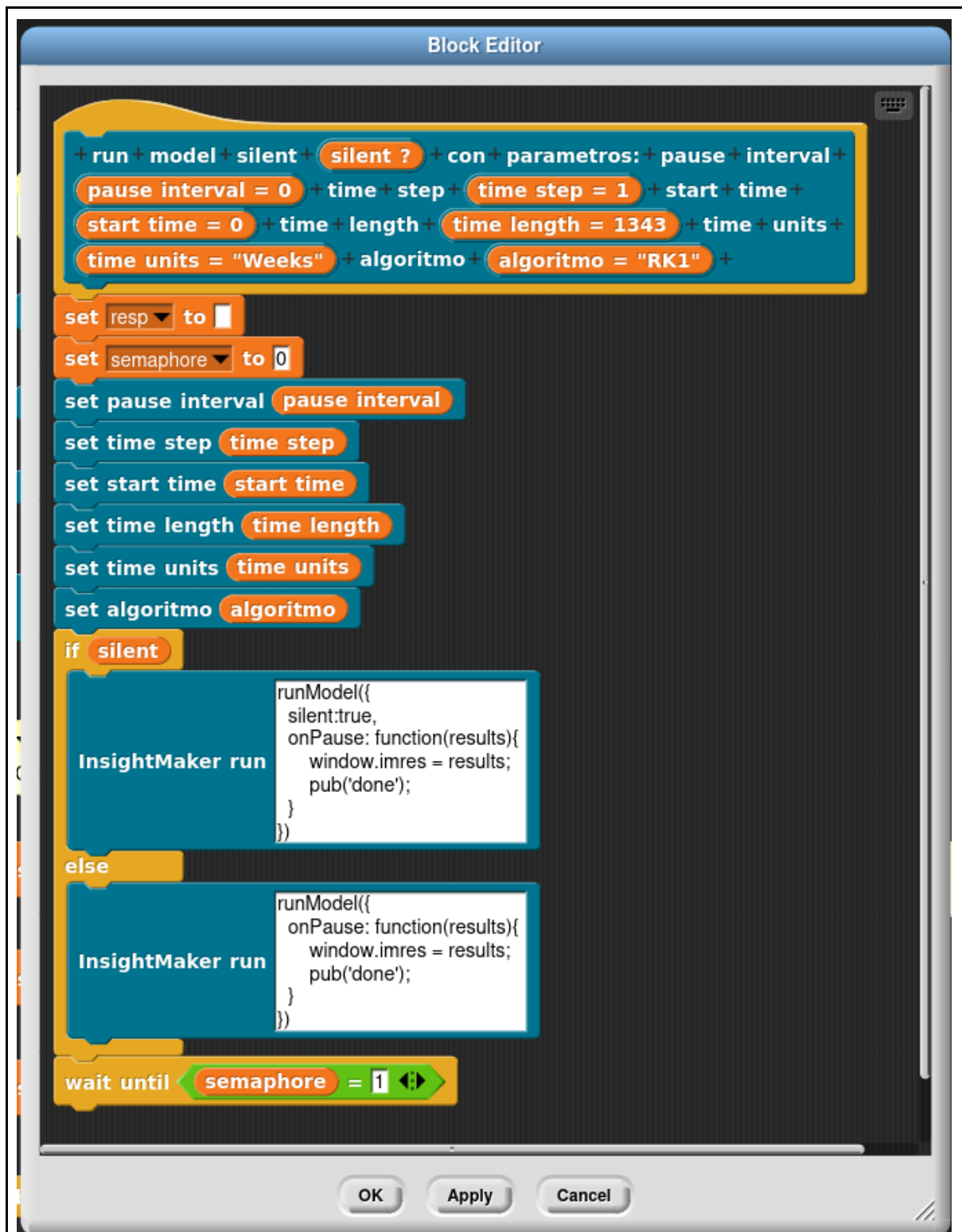
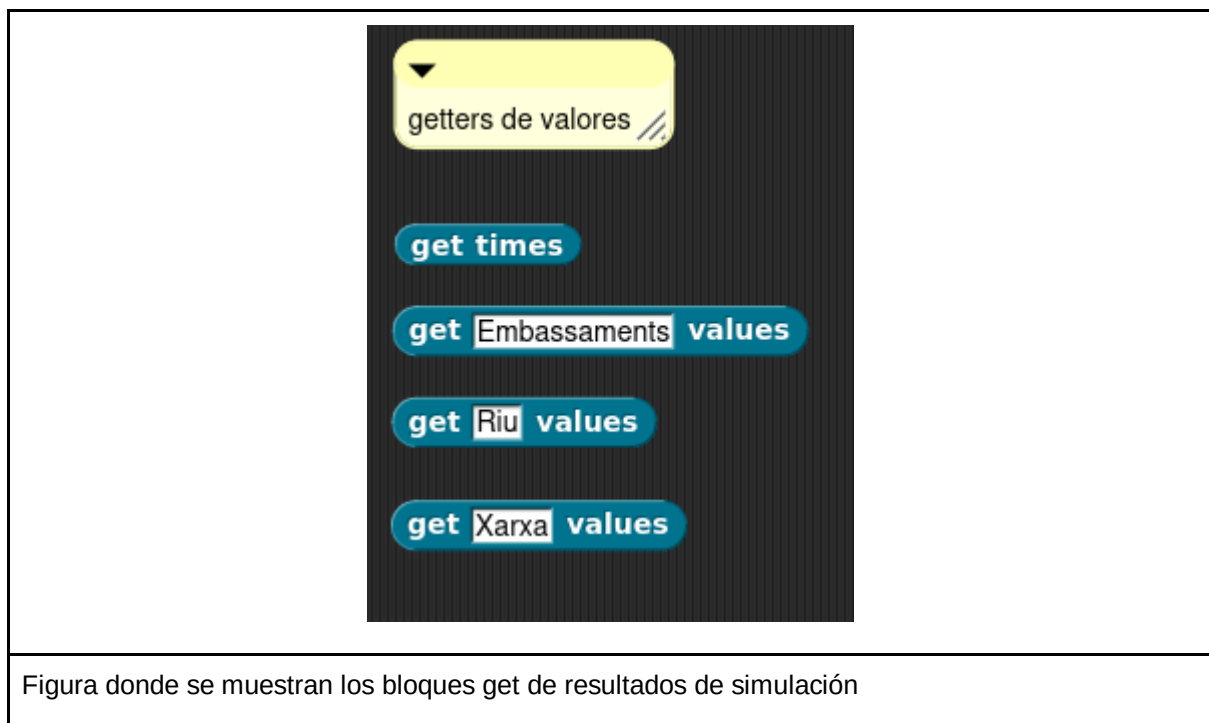


Figura donde se muestra la estructura interna del bloque

Podemos destacar que este bloque reutiliza los setters explicados anteriormente para establecer los parámetros antes de llamar a “runModel” de la API de InsightMaker. Además, permite con un toggle seleccionar la opción de *run silent*.

En tercer lugar, queda poder leer los resultados de la simulación, ya sea paso a paso, o de toda la ejecución. Esta funcionalidad se realiza con los siguientes bloques:



En este ámbito, nos hemos centrado principalmente en los valores de los *stocks*, ya que suelen ser los resultados deseados para análisis, aunque se podría expandir la implementación para acomodar otras entidades de la simulación como *flows*. Debajo adjuntamos la estructura interna del bloque, cabe destacar que la diferencia entre *get times* y *get (parámetro) values* radica simplemente en el uso del parámetro.

Adicionalmente, notamos que la funcionalidad de poder ver resultados intermedios de la ejecución se puede conseguir si se establece un *pause interval* positivo y se llama a “get _ values” cuando la simulación está pausada.

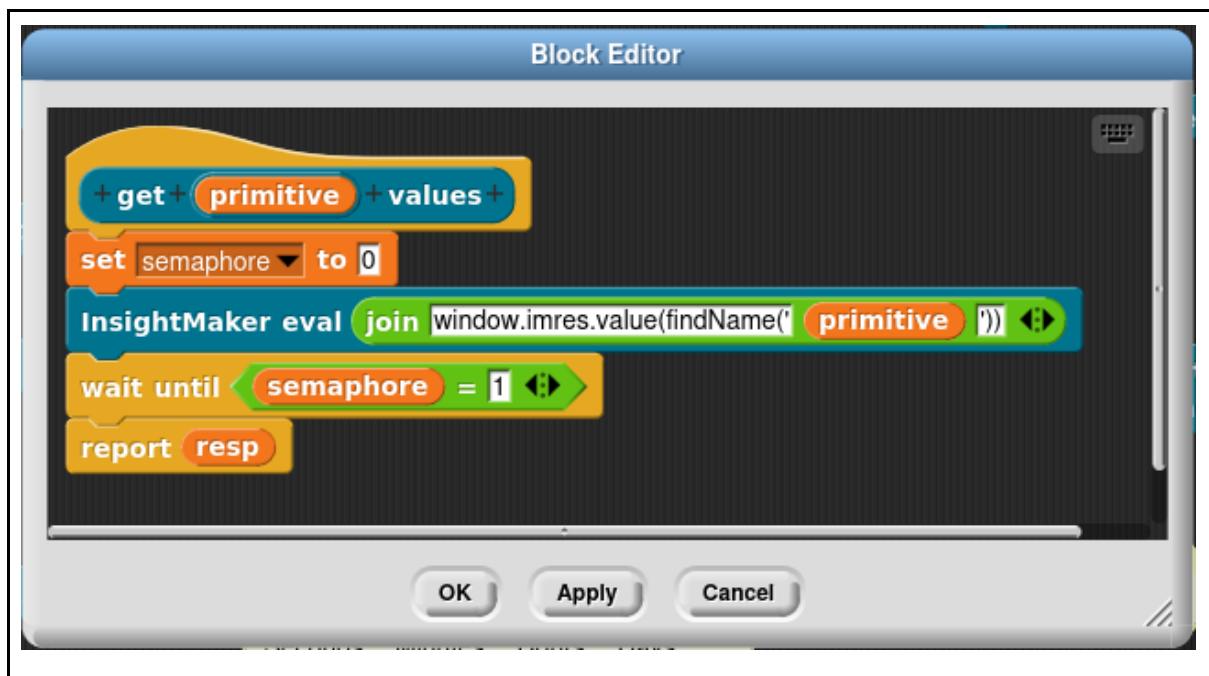


Figura donde se muestra la estructura interna del bloque “get _ values”

Por último, comprobamos el correcto funcionamiento de todos los bloques. Para ello, simplemente adjuntamos capturas de pantalla con el éxito de cada bloque.

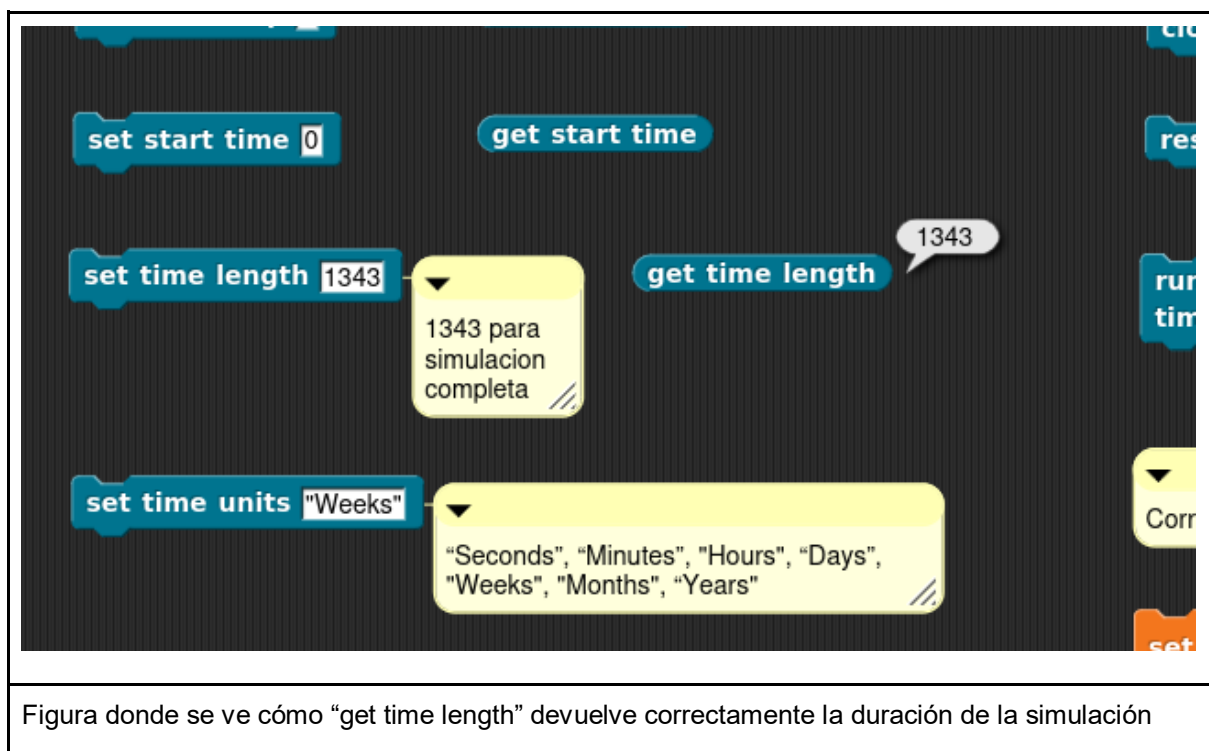
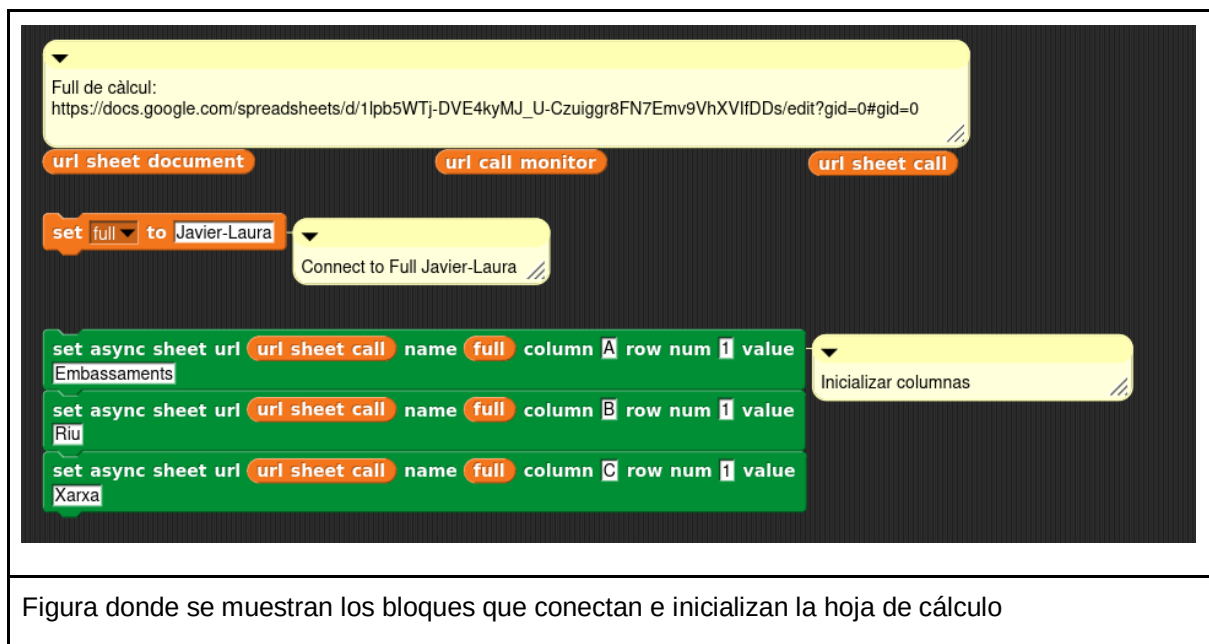


Figura donde se ve cómo “get time length” devuelve correctamente la duración de la simulación

3. Conexión con entorno de visualización

En este apartado explicaremos brevemente cómo hemos implementado los bloques necesarios para enviar los resultados a un entorno de visualización, en este caso una hoja de cálculo. Para ello, también adjuntamos capturas de pantalla para mejorar la legibilidad como en el apartado anterior.

Primero de todo, tenemos la conexión con la hoja de cálculo y la inicialización de las columnas donde se escribirán los datos.



Sin embargo, antes de empezar a escribir los datos, es vital inicializar las variables que guardan estos datos en Snap. Lo hacemos de esta manera para no tener que llamar constantemente a la API de InsightMaker.

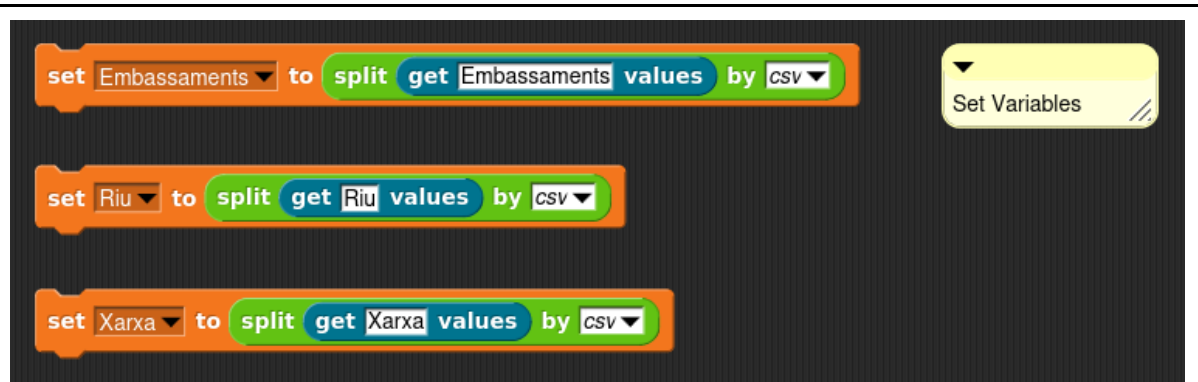


Figura donde se muestran los bloques que inicializan las variables que guardan los datos

A continuación, simplemente hace falta activar los bloques que escriben al *Google Sheets* usando los datos guardados en estas variables.

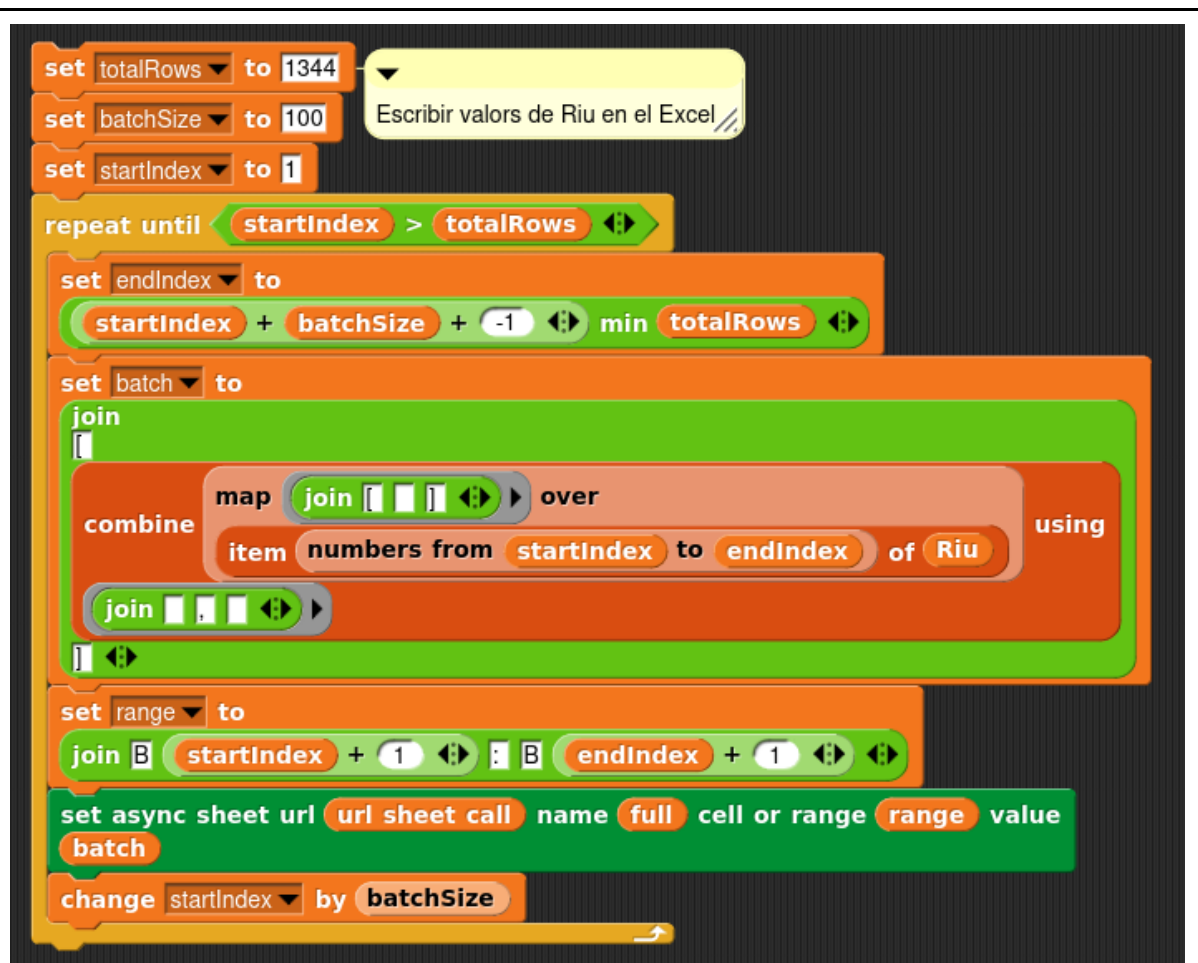


Figura que muestra el bloque que escribe los datos de Riu

Primeramente, los 2 bloques que tenemos disponibles para escribir en la hoja de cálculo tienen problemas para escribir todos los datos de forma directa.

El primer bloque que escribe un solo valor se descarta por simple hecho de que es terriblemente ineficiente para escribir los miles de valores que necesitamos escribir. Mientras que el otro, el que escribe un rango de valores, por las limitaciones de *Google Sheets* es incapaz de enviar todo el rango de valores a la vez. En conclusión, nos hemos visto obligados a separar los datos en *batches*, hemos escogido de tamaño 100 para simplificar, con el bloque que escribe por rangos.

El principio del bloque inicializa las variables que se usan, aparte de las que guardan los datos, y entra en un bucle que itera hasta que ha procesado todos los datos. “StartIndex” representa el índice inicial, mientras que “EndIndex” representa el índice final, este último se calcula como el mínimo entre “StartIndex” + “BatchSize” - 1 y “TotalRows”.

A continuación, el bloque formatea los datos para poderlos enviar a *Google Sheets* en formato JSON. Esto lo consigue con una serie de operaciones sobre la variable que contiene los datos. Estudiando en más detalle cómo se inicializa esta variable, nos damos cuenta de que usa el bloque “split”, por lo que la variable resultante es una lista.

De esta manera, el bloque “item numbers from StartIndex to EndIndex of _” recolecta los elementos de la lista desde el índice StartIndex hasta el índice EndIndex. Una vez tenemos estos elementos, usamos el bloque “map” para aplicar el bloque “join [_]”, donde _ se sustituye por cada elemento de la lista, para escribir los paréntesis.

Finalmente, usamos un último bloque “join [_]” para acabar de cerrar los paréntesis y de esta forma hemos acabado de formatear los datos para el envío.

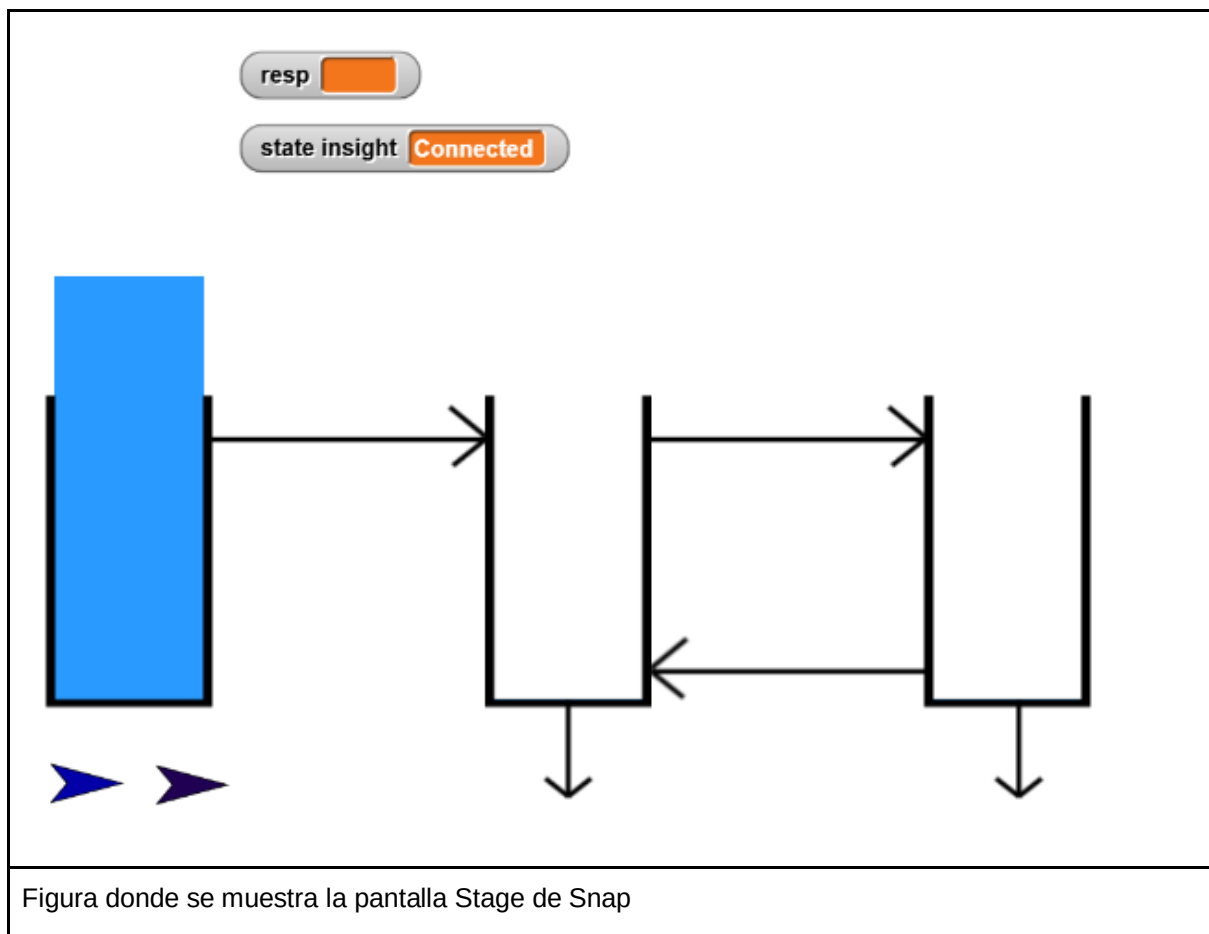
[illegible]

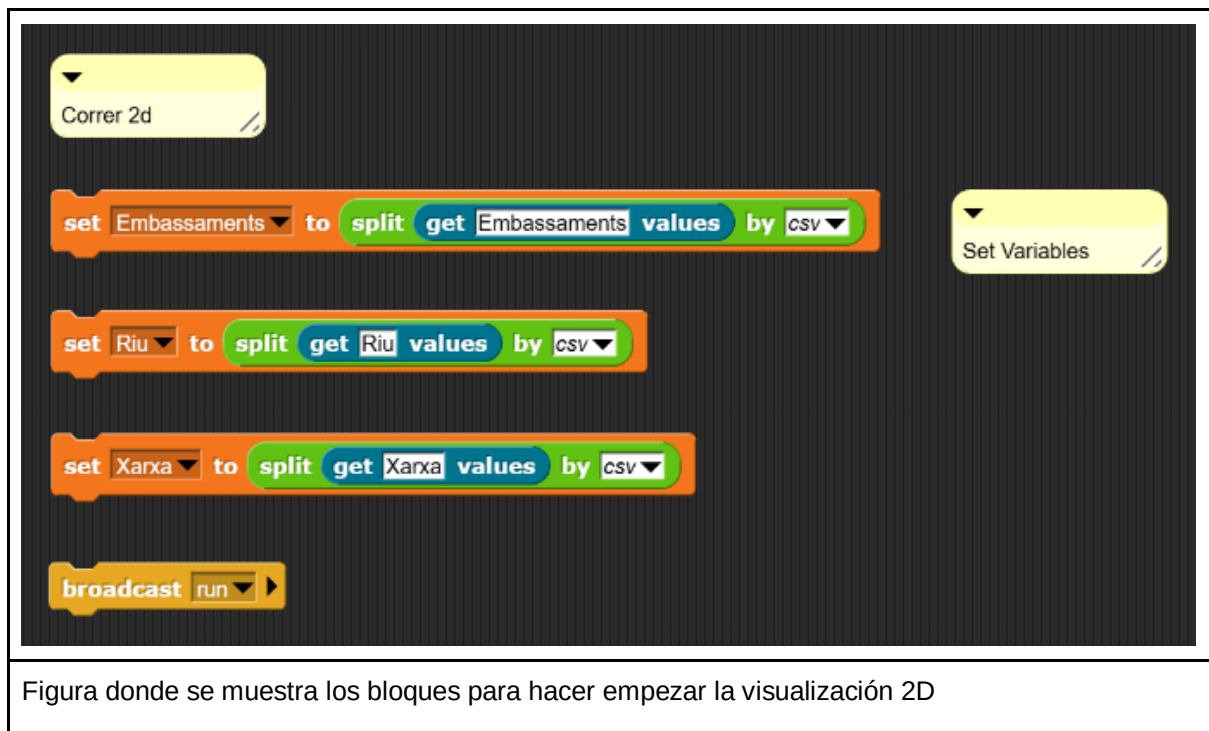
Figura donde se muestra el resultado de usar estos bloques

4. Visualización 2D

Finalmente, una vez se ha conectado Snap con Insight Maker y hecho correr la simulación, nos podemos centrar en la visualización de los datos del modelo en un formato 2d.

La inclusión de este elemento de visualización gráfico se hace dentro del propio Snap, utilizando su ventana de Stage, donde se puede ver en cada momento de la simulación en agua que contiene cada *stock*. Para poder hacer esto, se usan los gets de los *stocks* explicados en el apartado dos, se separa su resultado según el modelo .cvs y se coloca el resultado final en una variable del mismo nombre que el *stock*. Cuando se tienen las tres variables con los valores correctos, se hace el broadcast del mensaje “run”, el cual indicará a los bloques encargados de la visualización de cada *stock* que inicien su funcionamiento.





Hace falta destacar que, cada vez que se haga un run con valores distintos y se quiera ver la visualización 2D de este, se tendrá que volver a hacer los set de las tres variables de la figura.

Una vez hecho este broadcast, el mensaje “run” es recibido por los bloques en AiguaE, AiguaR y AiguaX, empezando estos su ejecución y mostrando el cambio de los valores en el tiempo en el Stage.



Figura donde se muestra los bloques para hacer que AiguaE, basada en Embassaments, se ejecute

La figura anterior sería el caso de Embassaments en AiguaE, para poder hacer Riu en AiguaR y Xarxa en AiguaX el bloque es casi igual, solamente hay que cambiar la variable Embassament per Riu o Xarxa y x,y en go to a las x,y correspondientes para cada uno, siendo estas: 0,-114 y 190,-114 respectivamente.

Acabada la visualización, se puede hacer que el sistema vuelva a sus valores predeterminados en el Stage. Aunque esto no es necesario, ya que si se vuelve a ejecutar el broadcast o uno de los *stocks* de manera separada, su ejecución no se basará en el estado actual del Stage, sino que volverá a mostrar los valores desde el inicio de manera correcta.



Figura donde se muestra los bloques para hacer que AiguaE, basada en Embassaments, vuelva a

su estado original



Figura donde se muestra los bloques para hacer que AiguaR, basada en Riu, vuelva a su estado original



Figura donde se muestra los bloques para hacer que AiguaX, basada en Xarxa, vuelva a su estado original

De esta manera podemos hacer que vayan a la posición x,y indicada y cambien de vuelta al disfraz de rectángulo azul con anchura de 65 y altura igual al primer elemento de su *stock*.

También sirve en el caso de accidentalmente mover alguna pieza en Stage, ya que permite devolverla a su posición original. Es debido a esto que no solamente los *stocks* tienen un bloque indicando donde se tiene que colocar.

A blue Scratch 'go to x: y:' block with the x-coordinate set to -190 and the y-coordinate set to -50.

Figura donde se muestra los bloques para hacer que Embassaments vuelva a su estado original

A blue Scratch 'go to x: y:' block with the x-coordinate set to 0 and the y-coordinate set to -50.

Figura donde se muestra los bloques para hacer que Riu vuelva a su estado original

A blue Scratch 'go to x: y:' block with the x-coordinate set to 190 and the y-coordinate set to -50.

Figura donde se muestra los bloques para hacer que Xarxa vuelva a su estado original

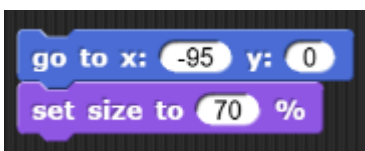
Two stacked Scratch blocks: a blue 'go to x: y:' block with x: -95 and y: 0, and a purple 'set size to %' block with the value 70.

Figura donde se muestra los bloques para hacer que Dessembasament vuelva a su estado original

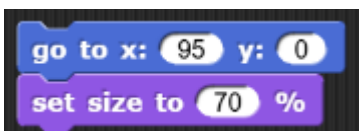
Two stacked Scratch blocks: a blue 'go to x: y:' block with x: 95 and y: 0, and a purple 'set size to %' block with the value 70.

Figura donde se muestra los bloques para hacer que Consum vuelva a su estado original

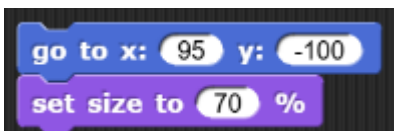
Two stacked Scratch blocks: a blue 'go to x: y:' block with x: 95 and y: -100, and a purple 'set size to %' block with the value 70.

Figura donde se muestra los bloques para hacer que Regeneracio vuelva a su estado original



Figura donde se muestra los bloques para hacer que AiguesResiduals vuelva a su estado original

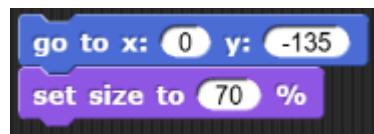
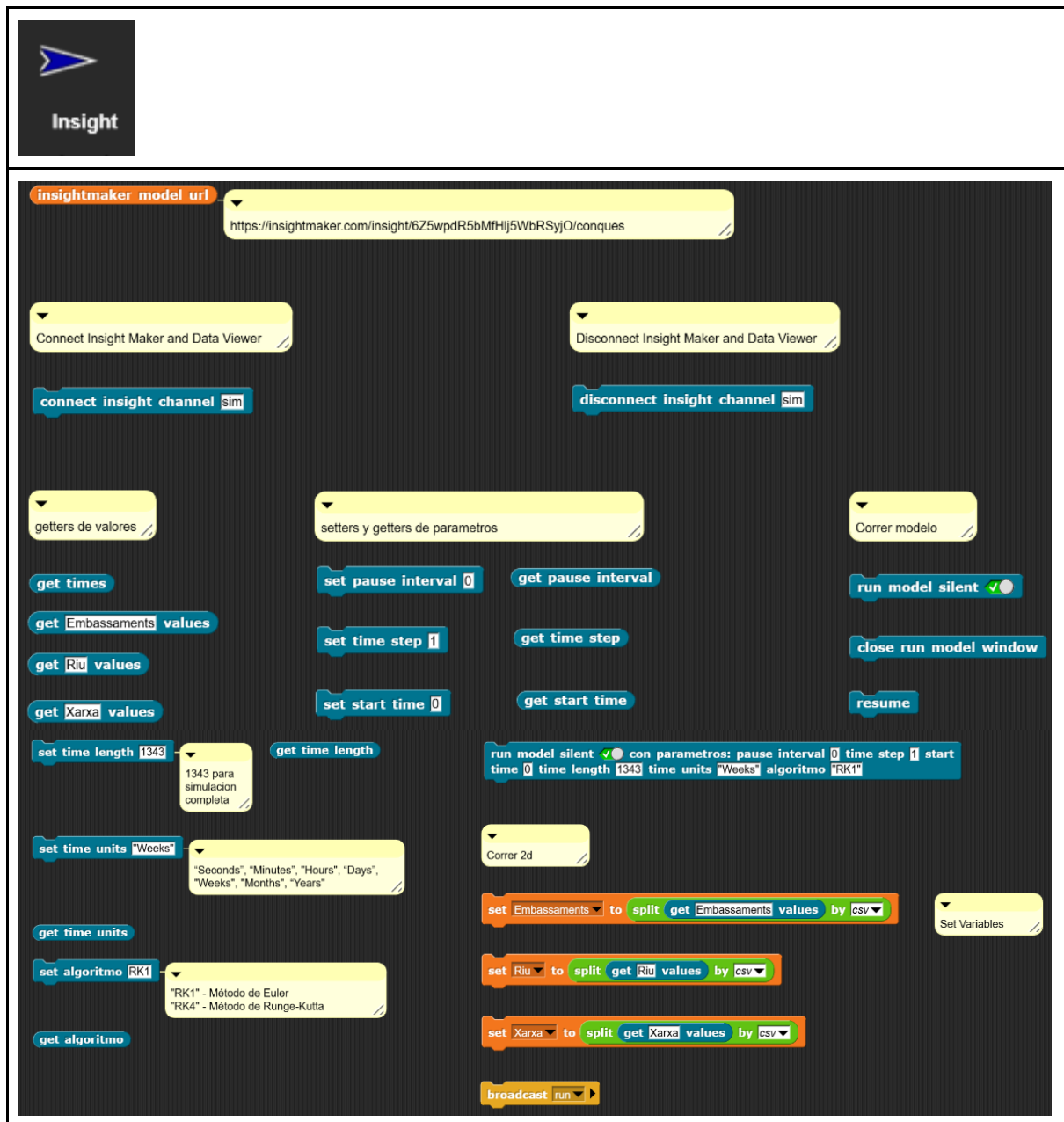


Figura donde se muestra los bloques para hacer que Desembocament vuelva a su estado original

5. Pantallas





GSheet

Full de càlcul:

https://docs.google.com/spreadsheets/d/1lpb5WTj-DVE4kyMJ_U-Czuigr8FN7Emv9VhXVIfDDs/edit?gid=0#gid=0

url sheet document

url call monitor

url sheet call

set full to Javier-Laura

Connect to Full Javier-Laura

set async sheet url url sheet call name full column A row num 1 value Embassaments

Inicializar columnas

set async sheet url url sheet call name full column B row num 1 value Riu

set async sheet url url sheet call name full column C row num 1 value Xarxa

set totalRows to 1344

set batchSize to 100

Escribir valores de Riu en el Excel

set startIndex to 1

repeat until startIndex > totalRows

set endIndex to

startIndex + batchSize + -1 min totalRows

set batch to

join

combine

map join [] over

item numbers from startIndex to endIndex of Riu

using

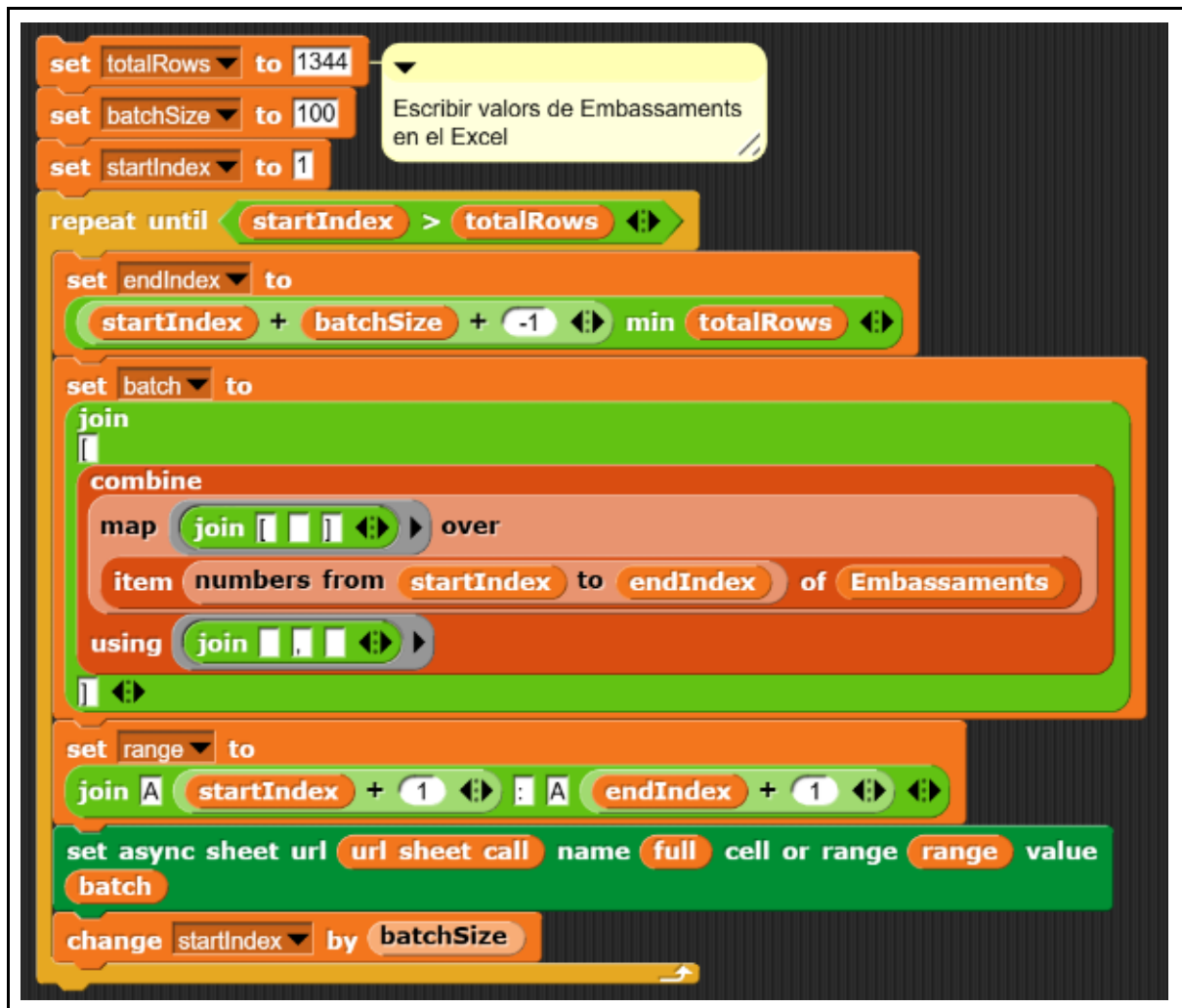
join []

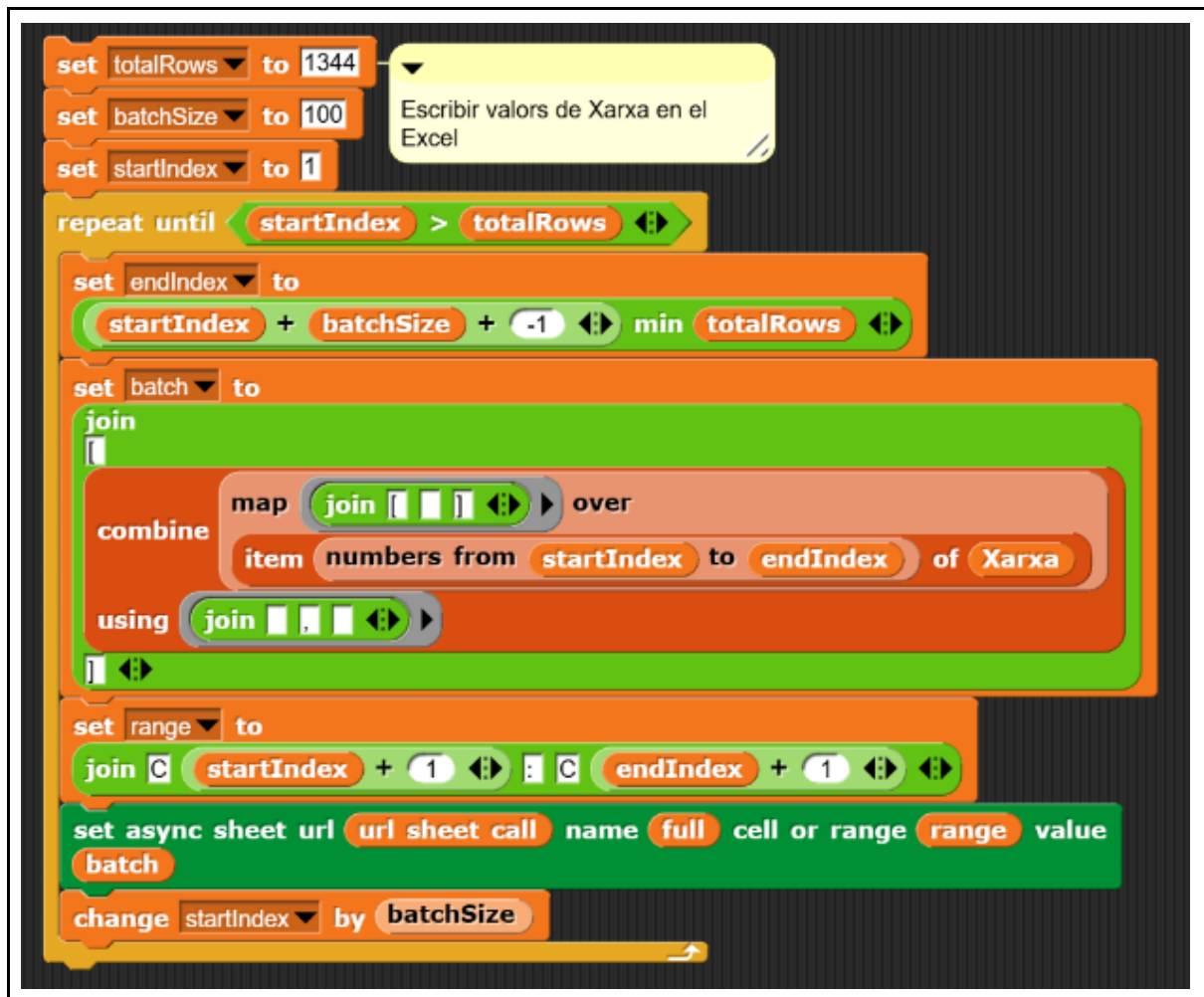
set range to

join B startIndex + 1 : B endIndex + 1

set async sheet url url sheet call name full cell or range range value batch

change startIndex by batchSize





AiguaE

☒ draggable

Correr 2d Embassaments

when I receive run

go to x: -190 y: -23

set width to 65

set tam-e to item 1 of Embassaments

for each item in Embassaments

set height to item / 2

switch to costume rectangle costume width: width height: height

change y by item - tam-e / 4

set tam-e to item

El valor real esta dividido entre 2

Restart 2d Embassaments

go to x: -190 y: -23

set width to 65

set height to item 1 of Embassaments / 2

switch to costume rectangle costume width: width height: height

Set Embassaments

set Embassaments to split get Embassaments values by csv

Riu

☒ draggable

go to x: 0 y: -50

AiguaR

☒ draggable

Correr 2d Riu

when I receive run

go to x: 0 y: -114

set width to 65

set tam-r to item 1 of Riu

for each item in Riu

set height to item / 2

switch to costume rectangle costume width: width height: height

change y by item - tam-r / 4

set tam-r to item

El valor real esta dividido entre 2

Restart 2d Riu

go to x: 0 y: -114

set width to 65

set height to item 1 of Riu / 2

switch to costume rectangle costume width: width height: height

Set Riu

set Riu to split get Riu values by csv

Xarxa

☒ draggable

go to x: 190 y: -50

AiguaX

☒ draggable

Correr 2d Xarxa

when I receive run

go to x: 190 y: -114

set width to 65

set tam-x to item 1 of Xarxa

for each item in Xarxa

set height to item / 2

switch to costume rectangle costume width: width height: height

change y by item - tam-x / 4

set tam-x to item

El valor real esta dividido entre 2

Restart 2d Xarxa

go to x: 190 y: -114

set width to 65

set height to item 1 of Xarxa / 2

switch to costume rectangle costume width: width height: height

Set Xarxa

set Xarxa to split get Xarxa values by csv

Dessembasament

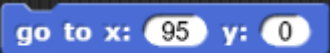
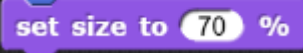
☒ draggable

go to x: -95 y: 0

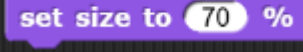
set size to 70 %

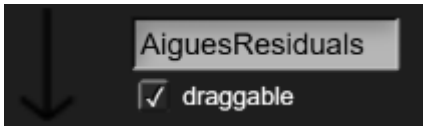
Consum

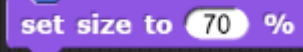
☒ draggable

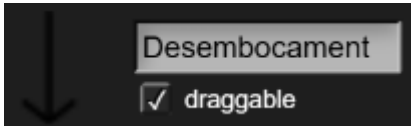



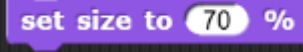

← Regeneracio
✓ draggable

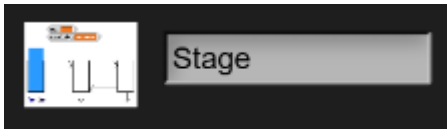




↓ AiguesResiduals
✓ draggable


↓ Desembocament
✓ draggable


 Stage

